

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«Новгородский государственный университет имени Ярослава
Мудрого»**

Политехнический институт
Кафедра информационных технологий и систем

ОТЧЁТ
по учебной практике

на тему:

**«ИИ-генерация и автоматическая оценка
рифмованной русской поэзии»**

по направлению подготовки 09.04.01 «Информатика и вычислительная
техника»,
профиль подготовки «Искусственный интеллект»

Руководитель:

_____ / _____
«____» _____ 2026 г.

Студент группы 5095:

_____ / С. Каримов
«____» _____ 2026 г.

Великий Новгород

2026

АННОТАЦИЯ

Отчёт по учебной практике посвящён разработке прототипа конвейера ИИ-генерации и автоматической оценки рифмованной русской поэзии. Объект исследования — методы генерации и автоматической оценки рифмованной русской поэзии; предмет — конвейер, объединяющий генерацию стиха большой языковой моделью с численной оценкой рифмы и отбором лучшего варианта (best-of-N).

Цель учебной практики — разработать прототип такого конвейера (приложение «Rhymer») и подготовить материалы исследования: обзор источников, план-график работы над ВКР, описание данных и спецификацию системы. В ходе практики выполнен аналитический обзор предметной области, составлен библиографический список, разработан двухлетний план-график с диаграммой Ганта, обоснован выбор данных, сформулированы математическая постановка задачи и метрики качества. Реализован прототип «Rhymer» на PySide6: генерация (best-of-N с самокоррекцией) и два взаимозаменяемых детектора рифмы — RPST и авторский алгоритм на потенциале Юкавы с фонетической векторизацией слогов. Научная новизна сосредоточена в авторском алгоритме оценки созвучия и способе векторизации слогов.

Область применения результатов — инструменты поддержки авторов и редактирования стихов, образование (стиховедение, фонетика), а также объективная оценка генеративных моделей на поэтическом материале.

Объём отчёта — 34 с., 5 таблиц, рисунков и листингов — в приложениях, 48 использованных источников, 2 приложения.

Ключевые слова: ГЕНЕРАЦИЯ ПОЭЗИИ, ОЦЕНКА РИФМЫ, СИЛЛАБО-ТОНИКА, БОЛЬШИЕ ЯЗЫКОВЫЕ МОДЕЛИ, ДЕТЕКТОР РИФМЫ, ПОТЕНЦИАЛ ЮКАВЫ, ФОНЕТИЧЕСКАЯ ВЕКТОРИЗАЦИЯ, BEST-OF-N, RUSSIAN POETRY SCANSION TOOL.

ABSTRACT

This report on the academic practice is devoted to the development of a prototype pipeline for AI-based generation and automatic evaluation of rhymed Russian poetry. The object of research is methods for generating and automatically evaluating rhymed Russian poetry; the subject is a pipeline that combines verse generation by a large language model with a numerical rhyme score and best-of-N selection of the best candidate.

The goal of the practice is to build a prototype of such a pipeline (the «Rhymer» application) and to prepare research materials: a literature review, a schedule of work on the master's thesis, a description of the data, and a system specification. A working prototype was implemented in PySide6: best-of-N generation with self-correction and two interchangeable rhyme detectors — RPST and an original algorithm based on the Yukawa potential with phonetic vectorization of syllables. The scientific novelty lies in the original consonance-scoring algorithm and the syllable vectorization method.

Keywords: POETRY GENERATION, RHYME EVALUATION, SYLLABIC-ACCENTUAL VERSE, LARGE LANGUAGE MODELS, RHYME DETECTOR, YUKAWA POTENTIAL, PHONETIC VECTORIZATION, BEST-OF-N, RUSSIAN POETRY SCANSION TOOL.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
Глава 1. Обзор источников и теоретические основы.....	7
1.1 Методология поиска источников.....	7
1.2 Аналитический обзор предметной области.....	7
1.3 Оформление по ГОСТ и работа с eLibrary.....	8
Глава 2. План-график работы над ВКР.....	10
2.1 Помесячный план первого курса.....	10
2.2 Помесячный план второго курса.....	10
2.3 Контрольные точки.....	11
Глава 3. Работа с данными.....	12
3.1 Поиск и обоснование.....	12
3.2 Описание наборов данных.....	12
3.3 Источники и лицензионная чистота.....	13
3.4 Предобработка.....	13
3.5 Протокол сбора данных.....	13
Глава 4. Спецификация системы и прототип.....	15
4.1 Математическая постановка.....	15
4.2 Метрики качества.....	15
4.3 Требования к окружению.....	16
4.4 Пайплайн данных.....	16
4.5 Практическая часть: прототип «Rhymer».....	16
4.6 Взаимодействие с ИИ: оценка — коррекция.....	17
ЗАКЛЮЧЕНИЕ.....	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	20
ПРИЛОЖЕНИЕ А. Диаграмма Ганта и интерфейс приложения «Rhymer».....	24
ПРИЛОЖЕНИЕ Б. Ключевые фрагменты исходного кода.....	30

ВВЕДЕНИЕ

Оценка качества рифмованного стиха — задача на стыке стиховедения, лингвистики и обработки естественного языка. Созвучие (рифма), стихотворный размер и благозвучие традиционно оцениваются человеком «на слух», что плохо масштабируется и слабо воспроизводимо. С появлением больших языковых моделей (LLM) генерация текста стала массовой, однако силлабо-тоническая русская поэзия по-прежнему даётся моделям тяжело: одновременно выдержать смысл, размер и рифму удаётся не всегда.

Актуальность работы имеет три измерения. Социальное — сохранение интереса к поэзии средствами современных технологий, мост между классической традицией и цифровым поколением. Государственное — поддержка культуры и русского языка как части культурного кода. Научное — систем искусственного интеллекта, которые пишут и объективно оценивают стихи на русском языке, крайне мало, что оставляет нишу для новизны.

Объект исследования — методы генерации и автоматической оценки рифмованной русской поэзии. Предмет исследования — конвейер «генерация — численная оценка рифмы — отбор лучшего варианта», пригодный для практического приложения и дальнейшего развития в рамках выпускной квалификационной работы (ВКР).

Цель учебной практики — разработать прототип конвейера и подготовить исследовательские материалы первого года обучения. Для достижения цели поставлены задачи:

- провести обзор источников и анализ современного состояния области (SOTA);
- составить план-график работы над ВКР и диаграмму Ганта;
- обосновать выбор данных, описать наборы и протокол их подготовки;
- сформулировать математическую постановку задачи и метрики качества;
- реализовать работающий прототип приложения «Rhymet».

Гипотеза исследования: численная оценка рифмы (в том числе собственным графовым алгоритмом) в связке с отбором лучшего из N сгенерированных вариантов и одним проходом самокоррекции повышает точность рифмовки по заданной схеме по сравнению с прямой генерацией без обратной связи.

Отчёт состоит из введения, четырёх глав, заключения, списка использованных источников и приложения. Глава 1 посвящена обзору источников, глава 2 — плану-графику работы, глава 3 — работе с данными, глава 4 — спецификации системы и прототипу.

Глава 1. Обзор источников и теоретические основы

1.1 Методология поиска источников

Поиск вёлся по базам и площадкам eLibrary.ru, arXiv, ACL Anthology, Scopus, Web of Science, Hugging Face и GitHub. Основные запросы: poetry generation, rhyme/meter detection, Russian scansion, controlled/constrained text generation, RLHF poetry, Russian LLM, poetry evaluation, «русская поэзия корпус». Критерии отбора — релевантность задаче (генерация или оценка стиха), наличие воспроизводимого метода, кода или датасета, приоритет работ 2016–2026 годов. Текущий объём — около 40 ключевых источников; для целевых 70–100 список расширяется отечественными работами по стиховедению и NLP с eLibrary.

1.2 Аналитический обзор предметной области

Генерация поэзии прошла путь от рекуррентных сетей с конечными автоматами для рифмы и метра (Hafez [1; 2], Deep-speare [3]) к токенов-свободным и управляемым моделям (ByGPT5 [4], PoeLM [7]) и далее к LLM общего назначения с управлением через промпт и декодирование. Для русского языка доступны открытые модели — ruGPT [9; 10], mGPT [11], YaLM [12], Vikhr [13], Saiga [14], GigaChat [15], оцениваемые бенчмарком MERA [16]. Узкое место — одновременное соблюдение смысла, размера и рифмы.

Для русского языка автоматическая оценка стиха фактически закрывается инструментом RPST (RussianPoetryScansionTool, И. Козиев) [17; 18]: расстановка ударений, классификация метра, детекция точных и неточных рифм, оценка «техничности». Этот инструмент используется в проекте как опорный внешний верификатор. Для неконтекстной детекции рифмы известны коллокационные методы (RhymeTagger [21; 22]).

Управляемая генерация (PPLM [25], FUDGE [26], grid-beam search [27], COLD [28], NeuroLogic [29]) даёт механизмы навязывания рифмы и метра без переобучения модели. Обучение с подкреплением с наградой за рифму (Generate-and-Revise [32]) и регуляризованный best-of-N [35] — прямая опора для направления ВКР, где rhyme-score выступает функцией вознаграждения [31; 34]. Метрики оценки поэзии слабо коррелируют между собой и с экспертами [36; 37], поэтому требуется комбинированный протокол (автометрики RPST совместно с экспертной оценкой).

Подготовка текста к анализу опирается на классический NLP-конвейер: токенизация и нормализация (приведение к нижнему регистру, удаление пунктуации), лемматизация и частеречная разметка, затем — векторизация. Классические векторные представления слов (Word2Vec [44], FastText [45]) и контекстные модели (BERT [46]) кодируют семантику; в настоящей работе аналогичная идея векторизации переносится с уровня слов на уровень слогов, но с фонетическими, а не семантическими признаками, что и лежит в основе авторского детектора рифмы (глава 4).

Тематически источники сгруппированы по направлениям: нейросетевая и LLM-генерация поэзии; русскоязычные языковые модели; автоматический анализ стиха (метр, ударение, рифма); контролируемая генерация; обучение с подкреплением; метрики и экспертная оценка; корпуса и датасеты. Полный библиографический список приведён в конце отчёта.

1.3 Оформление по ГОСТ и работа с eLibrary

Регистрация на eLibrary.ru обязательна (профиль автора, доступ к РИНЦ, поиск отечественных публикаций). Библиографические записи оформляются по ГОСТ Р 7.0.100–2018, текст отчёта — по ГОСТ 7.32–2017. Для ведения базы источников удобно использовать Zotero с CSL-стилем «Russian GOST R 7.0.100-2018»: источники добавляются браузерным коннектором (выходные данные по DOI/arXiv подтягиваются

автоматически), а список литературы вставляется и пересобирается плагином в текстовом редакторе.

Глава 2. План-график работы над ВКР

Период обучения в магистратуре — сентябрь 2025 — июнь 2027 (четыре семестра). Учебная практика первого года соответствует разработке прототипа конвейера и подготовке исследовательских материалов. Ниже приведён помесечный план по курсам и контрольные точки; диаграмма Ганта вынесена в приложение А.

2.1 Помесечный план первого курса

Таблица 1 — Помесечный план первого курса

Месяц	Этап	Результат
Сен 2025	Выбор направления исследования	Направление, научный руководитель
Окт 2025	Формулировка темы; старт обзора литературы	Формулировка темы ВКР
Окт–Дек 2025	Отечественная литература, регистрация eLibrary	Библиографический список v1
Ноя 2025	Постановка цели и задач; объект и предмет	Черновик введения
Дек 2025	Гипотеза; техническое задание	ТЗ на ВКР
Янв 2026	Прототип «Rhymer»: RPST + генератор + best-of-N	Работающее MVP-приложение
Фев–Мар 2026	Зарубежная литература; анализ SOTA	Список 70–100 ист. (до 06.03.2026)
Мар–Апр 2026	План-график и диаграмма Ганта	План (до 24.04.2026)
Апр–Май 2026	Поиск и обоснование данных; протокол	Описание данных (до 24.05.2026)
Май 2026	Спецификация: постановка, метрики, окружение	Спецификация (до 29.05.2026)
Июн 2026	Отчёт по практике; предзащита и защита	Зачёт по практике

2.2 Помесечный план второго курса

Таблица 2 — Помесечный план второго курса

Месяц	Этап	Результат
Сен 2026	Уточнение архитектуры; baseline-модель	Baseline + метрики
Сен–Ноя 2026	Разработка основной	Основная модель v1

	модели	
Окт 2026	Тезисы/статья на конференцию	Поданные тезисы
Ноя 2026–Янв 2027	Серия экспериментов; логи	Таблицы результатов
Янв 2027	Формализация задачи; метрики; ограничения	Формальная постановка
Фев–Мар 2027	Доработка модели; написание глав ВКР	Главы 1–3 ВКР
Апр 2027	Оформление по ГОСТ; антиплагиат	Свёрстанный текст ВКР
Апр 2027	Презентация; предзащита; рецензия	Отзыв и рецензия
Май–Июн 2027	Защита ВКР перед ГЭК	Диплом

2.3 Контрольные точки

- 06.03.2026 — расширенный список литературы (70–100) и анализ SOTA;
- 24.04.2026 — утверждённый план-график и диаграмма Ганта;
- 24.05.2026 — описание набора данных и протокол сбора;
- 29.05.2026 — спецификация (постановка, метрики, окружение, пайплайн);
- июнь 2026 — защита отчёта по учебной практике;
- апрель 2027 — предзащита, отзыв и рецензия;
- июнь 2027 — защита ВКР перед ГЭК.

Глава 3. Работа с данными

3.1 Поиск и обоснование

Требования к данным: русскоязычная силлабо-тоническая поэзия со строфами регулярной рифмовки; желательна разметка ударений, схемы рифмовки и метра; открытая лицензия; достаточный объём и разнообразие авторов и эпох, чтобы метрики не переобучались на одном стиле. Для оценки детектора рифмы нужна разметка — отсюда выбор корпусов Rifma [20] и ArsPoetica [19] (совместимы с RPST); для генерации и языкового разнообразия — крупные неразмеченные корпуса (stihi_ru [40], поэтический подкорпус НКРЯ [41], PULPO [42]); для кросс-языковых экспериментов — Gutenberg Poetry Corpus [43].

3.2 Описание наборов данных

Таблица 3 — Наборы данных — кандидаты

Датасет	Язык	Объём	Разметка	Лицензия
Rifma	RU	5002 фрагмента	ударения + схема рифмовки	см. репозиторий
ArsPoetica	RU	~8,5 тыс. стихов	ударения	см. карточку
stihi_ru	RU	крупный корпус	нет (сырой текст)	см. карточку
Поэтический подкорпус НКРЯ	RU	большой	метр/рифма/ строфика	условия НКРЯ
PULPO	мультияз.	>72–95 млн слов	нет	см. карточку
Gutenberg Poetry	EN	3,09 млн строк	нет	CC0
poems/ (проект)	RU	50 строф / 359 строк	нет	классика (PD)

Основной выбор: для обучения и валидации детектора рифмы — Rifma и ArsPoetica; для языкового разнообразия и генерации — stihi_ru (и НКРЯ

при доступе); для сравнения с зарубежным SOTA — PULPO и Gutenberg Poetry.

3.3 Источники и лицензионная чистота

Корпуса Козиева (Rifma, ArsPoetica) совместимы с RPST по формату разметки, что снижает рассогласование. НКРЯ — академический корпус с профессиональной стиховедческой разметкой (доступ по условиям корпуса). Датасеты Hugging Face используются согласно лицензиям в карточках. Тексты современных авторов (stihi.ru и т. п.) могут быть защищены авторским правом, поэтому приоритет отдаётся произведениям в общественном достоянии (классика) и датасетам с явной открытой лицензией; современные тексты используются только в агрегированном/метрическом виде или с разрешения.

3.4 Предобработка

- нормализация: UTF-8, нижний регистр, удаление пунктуации для детектора с сохранением исходного варианта для показа;
- сегментация на строфы (по пустой строке, фильтр ≥ 2 строк);
- дедупликация точных и near-duplicate строф;
- фильтрация по языку, длине, кодировке;
- разметка ударений (RPST) — хвосты/клаузулы; сверка с эталоном для размеченных датасетов;
- формирование примеров (строфа, целевая схема, метки рифмующихся пар);
- разбиение train/val/test со стратификацией по схеме и размеру, фиксация random seed и версии датасета.

3.5 Протокол сбора данных

- источники и права: фиксировать происхождение и лицензию каждого текста;

- сбор: предпочитать готовые датасеты; при парсинге соблюдать robots.txt и хранить дату выгрузки;
- очистка: нормализация, дедупликация, отсев не-русских и битых фрагментов;
- разметка: автоматическая (RPST) + выборочная ручная проверка ≥ 2 аннотаторами, согласие по Cohen's κ ;
- версионирование: версия, хэш и описание изменений; неизменяемые сплиты;
- метаданные: источник, лицензия, схема, размер, метки;
- хранение: сырые и обработанные данные отдельно, крупные файлы вне git.

Глава 4. Спецификация системы и прототип

4.1 Математическая постановка

Параметры запроса $\theta = (t, p, L, s)$: тема t , размер p (ямб, хорей и т. д.), число строк L , схема рифмовки s (например, АВАВ). Стихотворение $y = (\ell_1, \dots, \ell_L)$ — последовательность строк; строка разбивается на слоги. Генератор G_θ — языковая модель, $y \sim G_\theta(\cdot|\theta)$. Детектор рифмы D отображает (y, s) в набор метрик m .

Качество кандидата оценивается ранжирующим баллом $R(y, \theta) = 0,6 \cdot \rho(y) + 0,4 \cdot \min(\max(q(y), 0), 1)$, где $\rho(y)$ — доля зарифмованных слогов, $q(y)$ — оценка поэтичности RPST. Для детектора Юкавы $R(y, \theta) = \rho(y)$. В режиме best-of- N генерируются N кандидатов и выбирается $y^* = \operatorname{argmax} R(y, \theta)$. При точности по схеме $A(y^*, s) < 1$ строится текстовый фидбэк и запрашивается одна правка (самокоррекция); исправленный вариант также участвует в отборе.

Граф связей (ветвь Юкавы): узлы — слоги, сила связи $S_{ab} = \operatorname{sim}(a, b) \cdot K_\lambda(d_{ab}) \cdot |E_a \cdot E_b|/\eta$, где ядро экранирования $K_\lambda(d) = e^{(-\lambda d)}/d$, d_{ab} — манхэттенское расстояние по решётке (строка, позиция), sim — фонетическое созвучие слогов, E — «энергия» (масса) слога, η — нормировка. После резонанса $S'_{ab} = S_{ab} + \gamma \cdot (1/N) \cdot \sum_c S_{ac} \cdot S_{bc}$ и нормировки берутся рёбра $S'_{ab} \geq \beta \cdot \max S'$; компоненты связности образуют группы рифм.

4.2 Метрики качества

Таблица 4 — Метрики качества

Метрика	Определение	Назначение
rhyme_accuracy	доля верно срифмованных пар по схеме	основная для генерации
rhyme_percent	доля слогов в подсвеченных рифмах	ранжирование, наглядность
RPST score	оценка поэтичности по RPST	компонент балла R

meter accuracy	доля строк с корректным размером	контроль размера
precision/recall групп	TP/(TP+FP), TP/(TP+FN)	валидация детектора
корреляция с экспертом	Spearman ρ_s	согласие с человеком

4.3 Требования к окружению

Таблица 5 — Требования к окружению

Компонент	Значение
ОС	Linux (разработка); кроссплатформенно (PySide6)
Python	3.12 (виртуальное окружение)
GUI	PySide6 6.11
Анализ стиха	russian_scansion (RPST) 1.0.22 + модели
Научный стек	numpy, scipy, matplotlib, torch 2.10
LLM SDK	anthropic (Claude) и/или openai (Gemini/DeepSeek)
Архитектура	MVP (Model/View/Presenter), задачи в QThread

4.4 Пайплайн данных

Сбор корпуса — нормализация (приведение к нижнему регистру, удаление пунктуации, разбиение на строки и слова) — дедупликация и фильтрация (длина строф, язык, кодировка) — разметка ударений (RPST) и выделение хвостов/клаузул — формирование примеров (строфа, схема, метки рифмующихся пар) — разбиение train/val/test со стратификацией — оценка детектором и (для генерации) best-of-N с самокоррекцией — логи экспериментов для таблиц ВКР.

4.5 Практическая часть: прототип «Rhymer»

Прототип реализован как десктоп-приложение на PySide6 по паттерну MVP (Model/View/Presenter); тяжёлые задачи (загрузка моделей RPST, генерация, оценка) выполняются в отдельном потоке, интерфейс не блокируется. Генерация мультипровайдерная (Claude, Gemini, DeepSeek) по схеме best-of-N с одним проходом самокоррекции по фидбэку детектора.

Реализованы два взаимозаменяемых детектора рифмы с единым интерфейсом.

Первый детектор — обёртка над RPST: разметка ударений, определение размера и схемы рифмовки, оценка поэтичности, сравнение рифмующихся хвостов и расчёт точности по схеме. Второй — авторский алгоритм на потенциале Юкавы: стихотворение рассматривается как поток волн, сила созвучия слогов экранируется расстоянием, эффект резонанса усиливает созвучные цепочки, а группы рифм выделяются как компоненты связности графа. Ядро схожести слогов — фонетическая векторизация: каждый слог кодируется вектором артикуляционных признаков (отдельно блок гласной и блок согласных), а близость измеряется косинусом, так что основной вклад дают гласные, а согласные добавляют созвучие (аллитерацию). Интерфейс приложения, настройки провайдера и сохранение сессий показаны в приложении А.

4.6 Взаимодействие с ИИ: оценка — коррекция

Хотя тема ВКР относится к искусственному интеллекту, ядро работы — математика оценки рифмы; именно численные метрики направляют исправление текста. Взаимодействие с языковой моделью построено как цикл с обратной связью:

- генерация: модель выдаёт N вариантов стиха на тему (best-of-N);
- оценка: детектор рифмы численно оценивает каждый вариант (RPST или Юкава);
- разметка: по результату оценки помечаются окончания строк — какие рифмуются (оставить), а какие нет (переписать);
- коррекция: модели передаётся этот разбор и запрос переписать только плохо рифмующиеся строки, сохранив удачные рифмы, и проверить читаемость (естественный порядок слов) и форму (размер, число строк, схема).

Таким образом языковая модель выступает инструментом, а решения о том, что исправлять, принимаются на основе численных метрик качества рифмы. Это отличает подход от простого промптинга и делает результат воспроизводимым и измеримым.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики выполнены все поставленные задачи. Проведён обзор источников и анализ современного состояния области, составлен категоризированный библиографический список. Разработан двухлетний план-график работы над ВКР с диаграммой Ганта и контрольными точками. Обоснован выбор данных, описаны наборы и протокол их подготовки с учётом лицензионной чистоты. Сформулированы математическая постановка задачи, метрики качества, требования к окружению и пайплайн данных.

Главный практический результат — работающий прототип приложения «Rhumer»: генерация стиха большой языковой моделью по схеме best-of-N с самокоррекцией и два взаимозаменяемых детектора рифмы (RPST и авторский алгоритм на потенциале Юкавы с фонетической векторизацией слогов). Научная новизна сосредоточена в авторском алгоритме оценки созвучия и способе векторизации слогов.

Направления дальнейшей работы в рамках ВКР: дообучение и обучение с подкреплением генеративной модели с rhyme-score в качестве награды, расширение и разметка датасета, развитие приложения в полноценный редактор стихов. Гипотеза исследования на прототипе подтверждается: отбор лучшего из N вариантов с самокоррекцией повышает точность рифмовки по сравнению с прямой генерацией.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ghazvininejad M., Shi X., Choi Y., Knight K. Generating Topical Poetry // EMNLP 2016. URL: <https://aclanthology.org/D16-1126/>.
2. Ghazvininejad M., Shi X., Priyadarshi J., Knight K. Hafez: an Interactive Poetry Generation System // ACL 2017 (Demo). URL: <https://aclanthology.org/P17-4008/>.
3. Lau J. H., Cohn T., Baldwin T., Brooke J., Hammond A. Deep-speare: A Joint Neural Model of Poetic Language, Meter and Rhyme // ACL 2018. arXiv:1807.03491. URL: <https://arxiv.org/pdf/1807.03491>.
4. Belouadi J., Eger S. ByGPT5: End-to-End Style-conditioned Poetry Generation with Token-free Language Models // ACL 2023 Findings. arXiv:2212.10474. URL: <https://arxiv.org/abs/2212.10474>.
5. Lee S. W. et al. GPoeT-2: A GPT-2 Based Poem Generator. arXiv:2205.08847. URL: <https://arxiv.org/abs/2205.08847>.
6. Wang J., Zhang X., Zhou Y., Suh C., Rudin C. There Once Was a Really Bad Poet, It Was Automated but You Didn't Know It // TACL 2021. arXiv:2103.03775. URL: <https://arxiv.org/abs/2103.03775>.
7. Ormazabal A., Artetxe M., Soroa A., Labaka G., Agirre E. PoeLM: A Meter- and Rhyme-Controllable Language Model for Unsupervised Poetry Generation // EMNLP 2022 Findings. arXiv:2205.12206. URL: <https://arxiv.org/abs/2205.12206>.
8. Belouadi J., Eger S. Let the Poem Hit the Rhythm: Beat-Aligned Poetry Generation. arXiv:2406.10174. URL: <https://arxiv.org/html/2406.10174>.
9. Zmitrovich D. et al. A Family of Pretrained Transformer Language Models for Russian. arXiv:2309.10931. URL: <https://github.com/ai-forever/ru-gpts>.
10. AI Forever (Sber). ruGPT-3.5-13B. Hugging Face. URL: <https://huggingface.co/ai-forever/ruGPT-3.5-13B>.
11. Shliazhko O. et al. mGPT: Few-Shot Learners Go Multilingual. arXiv:2204.07580. URL: <https://arxiv.org/abs/2204.07580>.

12. Yandex. YaLM-100B. GitHub, 2022. URL: <https://github.com/yandex/YaLM-100B>.
13. Nikolich A. et al. Vikhr: Open-Source Instruction-Tuned LLMs for Russian. arXiv:2405.13929. URL: <https://arxiv.org/abs/2405.13929>.
14. Gusev I. saiga_llama3_8b. Hugging Face. URL: https://huggingface.co/IlyaGusev/saiga_llama3_8b.
15. Mukhamedshina D. et al. GigaChat Family: Efficient Russian Language Modeling Through MoE. arXiv:2506.09440. URL: <https://arxiv.org/abs/2506.09440>.
16. Fenogenova A. et al. MERA: A Comprehensive LLM Evaluation in Russian. arXiv:2401.04531. URL: <https://arxiv.org/abs/2401.04531>.
17. Koziev I. Automated Evaluation of Meter and Rhyme in Russian Generative and Human-Authored Poetry. arXiv:2502.20931, 2025. URL: <https://arxiv.org/abs/2502.20931>.
18. Koziev I. RussianPoetryScansionTool. GitHub. URL: <https://github.com/Koziev/RussianPoetryScansionTool>.
19. Koziev I. ArsPoetica. Hugging Face. URL: <https://huggingface.co/datasets/inkoziev/ArsPoetica>.
20. Koziev I. Rifma. GitHub. URL: <https://github.com/Koziev/Rifma>.
21. Plecháč P. A Collocation-Driven Method of Discovering Rhymes // Taming the Corpus. Springer, 2018. URL: <https://github.com/versotym/rhymetagger>.
22. Plecháč P. et al. Training Data Size Sensitivity in Unsupervised Rhyme Recognition. arXiv:2604.08156. URL: <https://arxiv.org/abs/2604.08156v1>.
23. Koziev I. rupostagger — POS Tagger for Russian. GitHub. URL: <https://github.com/Koziev/rupostagger>.
24. Koziev I. rutokenizer. GitHub. URL: <https://github.com/Koziev/rutokenizer>.
25. Dathathri S. et al. Plug and Play Language Models // ICLR 2020. arXiv:1912.02164. URL: <https://arxiv.org/abs/1912.02164>.

26. Yang K., Klein D. FUDGE: Controlled Text Generation With Future Discriminators // NAACL 2021. arXiv:2104.05218. URL: <https://arxiv.org/abs/2104.05218>.
27. Hokamp C., Liu Q. Lexically Constrained Decoding Using Grid Beam Search. arXiv:1704.07138. URL: <https://arxiv.org/abs/1704.07138>.
28. Qin L. et al. COLD Decoding: Energy-based Constrained Text Generation. arXiv:2202.11705. URL: <https://arxiv.org/pdf/2202.11705>.
29. Lu X. et al. NeuroLogic Decoding. arXiv:2010.12884. URL: <https://arxiv.org/pdf/2010.12884v1>.
30. Manurung R., Ranta A. et al. «Poetic» Statistical Machine Translation: Rhyme and Meter // EMNLP 2010. URL: <https://aclanthology.org/D10-1016.pdf>.
31. Huang S. et al. The N+ Implementation Details of RLHF with PPO (TL;DR). arXiv:2403.17031. URL: <https://arxiv.org/pdf/2403.17031>.
32. Casas N. et al. Generate and Revise: Reinforcement Learning in Neural Poetry. arXiv:2102.04114. URL: <https://arxiv.org/abs/2102.04114>.
33. Casas N. et al. Creative Data Generation: A Review Focusing on Text and Poetry. arXiv:2305.08493. URL: <https://arxiv.org/pdf/2305.08493>.
34. Lee H. et al. RLAIIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback. arXiv:2309.00267. URL: <https://arxiv.org/html/2309.00267v3>.
35. Regularized Best-of-N Sampling with MBR Objective for Language Model Alignment. arXiv:2404.01054. URL: <https://arxiv.org/abs/2404.01054>.
36. Belouadi J., Eger S. Evaluating Diversity in Automatic Poetry Generation. arXiv:2406.15267. URL: <https://arxiv.org/html/2406.15267v1>.
37. POEMetric: The Last Stanza of Humanity. arXiv:2604.03695. URL: <https://arxiv.org/html/2604.03695v1>.
38. Introducing Aspects of Creativity in Automatic Poetry Generation. arXiv:2002.02511. URL: <https://arxiv.org/pdf/2002.02511>.
39. Manurung R. et al. Autonomous Haiku Generation. arXiv:1906.08733. URL: <https://arxiv.org/pdf/1906.08733>.

40. Gusev I. stihi_ru. Hugging Face. URL: https://huggingface.co/datasets/IlyaGusev/stihi_ru.
41. Поэтический подкорпус Национального корпуса русского языка (НКРЯ). URL: <https://ruscorpora.ru/page/corpus-poetic-index/>.
42. PULPO: a multilingual poetry corpus. Hugging Face. URL: <https://huggingface.co/datasets/linhd-postdata/pulpo>.
43. Parrish A. Gutenberg Poetry Corpus (CC0). GitHub. URL: <https://github.com/aparrish/gutenberg-poetry-corpus>.
44. Mikolov T., Chen K., Corrado G., Dean J. Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781. URL: <https://arxiv.org/abs/1301.3781>.
45. Bojanowski P., Grave E., Joulin A., Mikolov T. Enriching Word Vectors with Subword Information // TACL 2017. arXiv:1607.04606. URL: <https://arxiv.org/abs/1607.04606>.
46. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // NAACL 2019. arXiv:1810.04805. URL: <https://arxiv.org/abs/1810.04805>.
47. ГОСТ Р 7.0.100–2018. Библиографическая запись. Библиографическое описание. М.: Стандартинформ, 2018.
48. ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. Структура и правила оформления. М.: Стандартинформ, 2017.

ПРИЛОЖЕНИЕ А. Диаграмма Ганта и интерфейс приложения «Rhymer»

(справочное)

Ниже приведены диаграмма Ганта плана работы над ВКР (глава 2) и экранные снимки прототипа «Rhymer» — десктоп-приложения на PySide6: генерация стиха, его улучшение по обратной связи детектора и оценка рифмы двумя взаимозаменяемыми детекторами (RPST и авторский алгоритм на потенциале Юкавы).

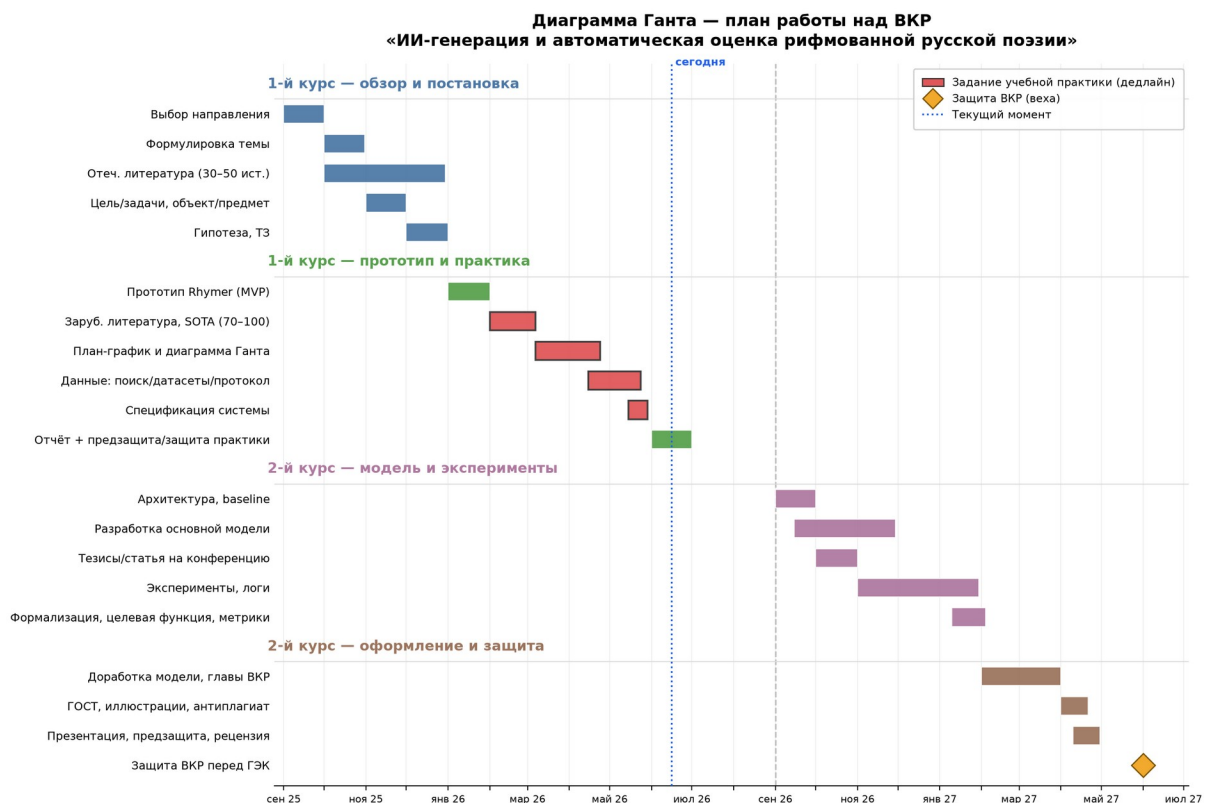


Рисунок А.1 — Диаграмма Ганта плана работы над ВКР (красным выделены задания учебной практики с дедлайнами; ромб — защита ВКР)

Исходный текст диаграммы в формате mermaid и помесечный план — в репозитории проекта (docs/practice/02_plan_gantt.md).

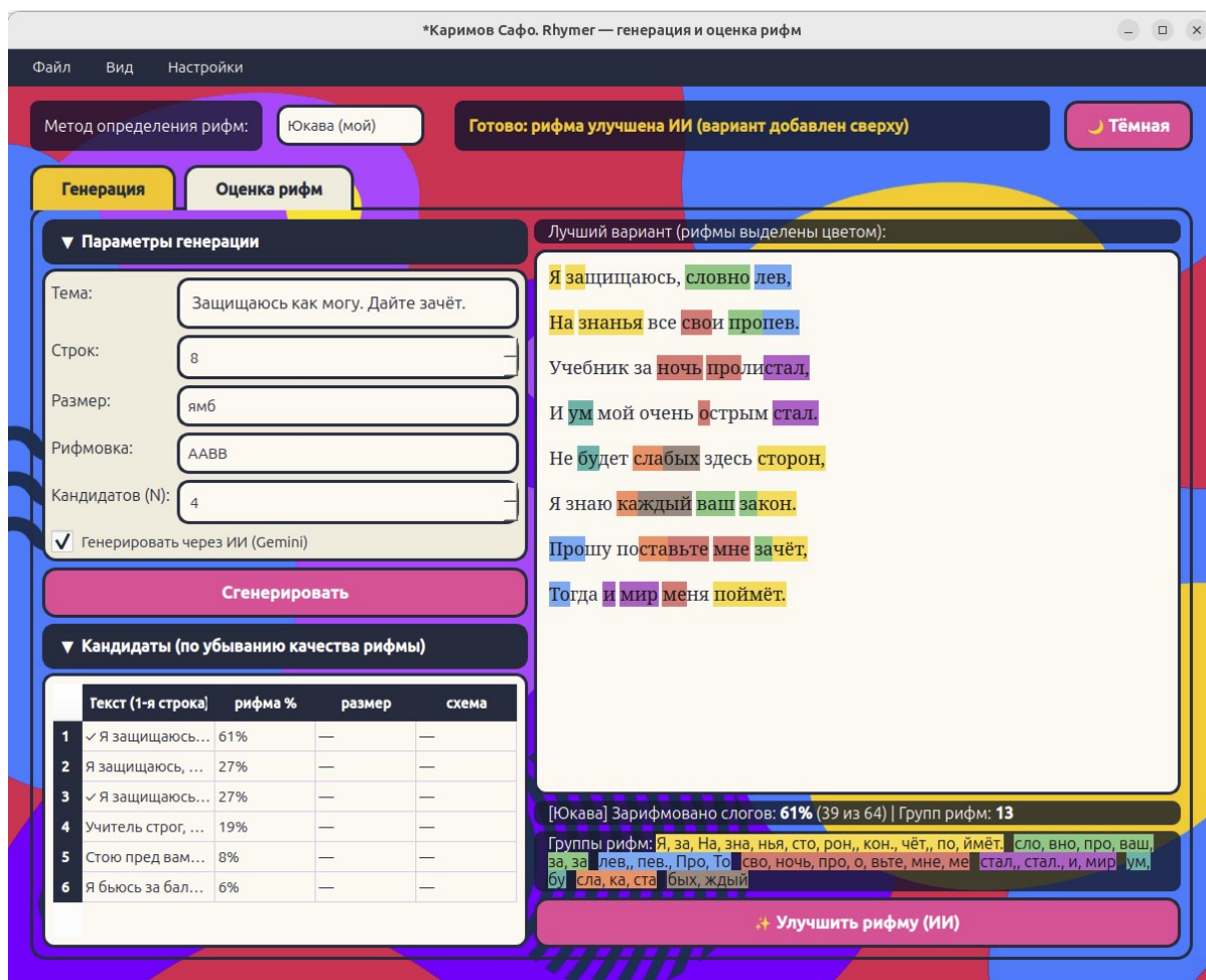


Рисунок А.2 — Вкладка «Генерация»: тема, размер, схема рифмовки и сгенерированный стих

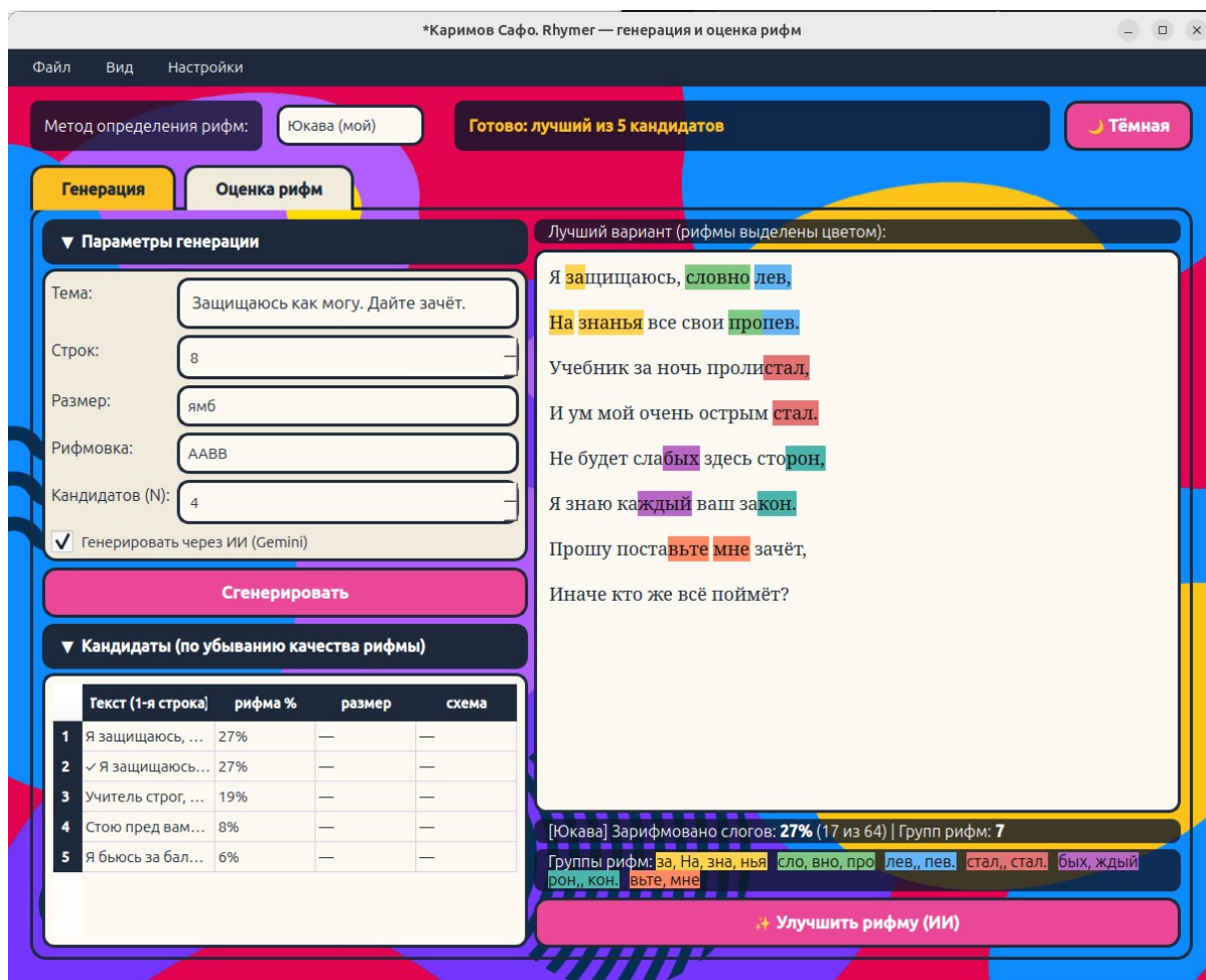


Рисунок А.3 — Улучшение стиха: самокоррекция по разбору рифмы (best-of-N с одним проходом правки)

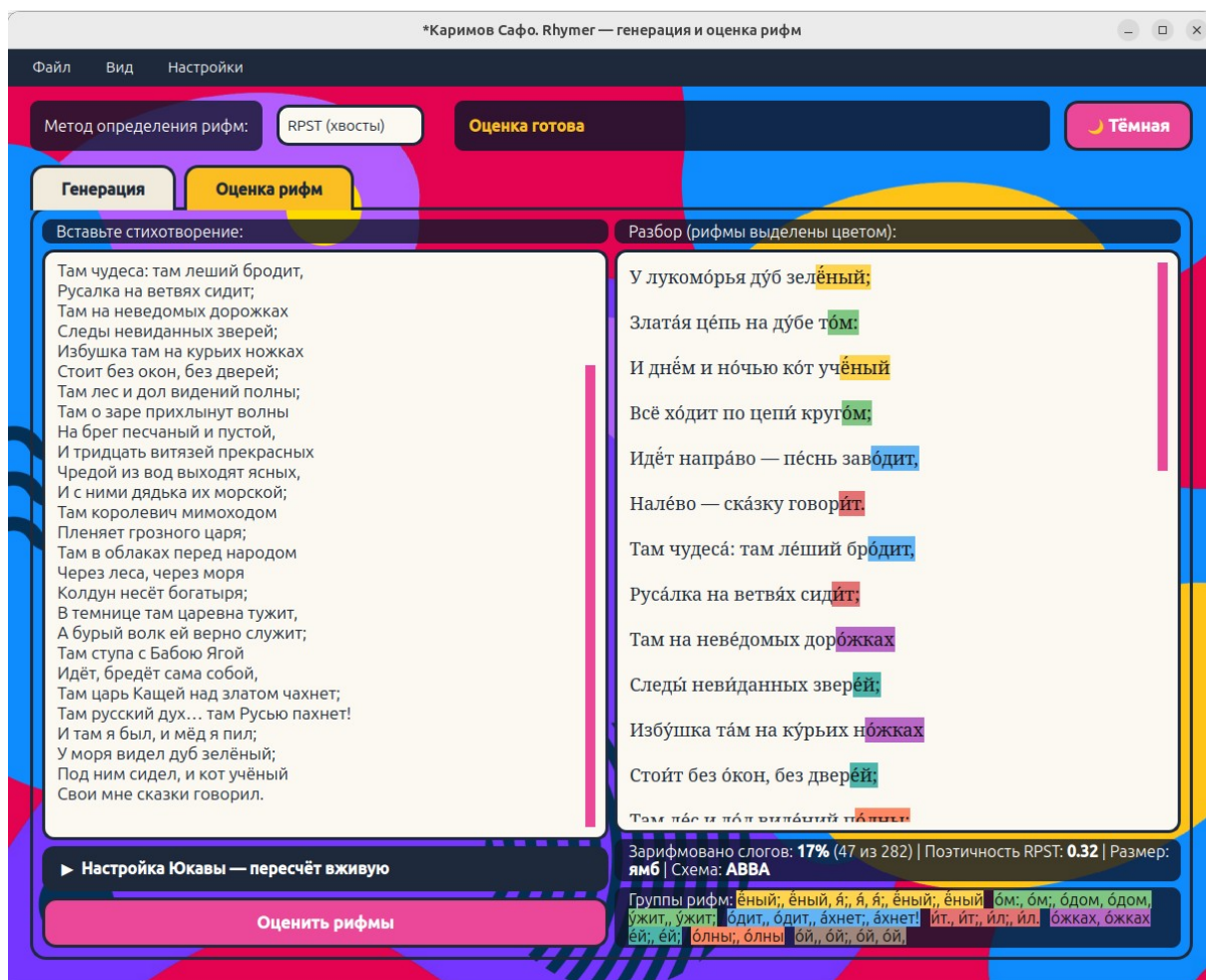


Рисунок А.4 — Оценка рифмы детектором RPST с подсветкой рифмующихся окончаний

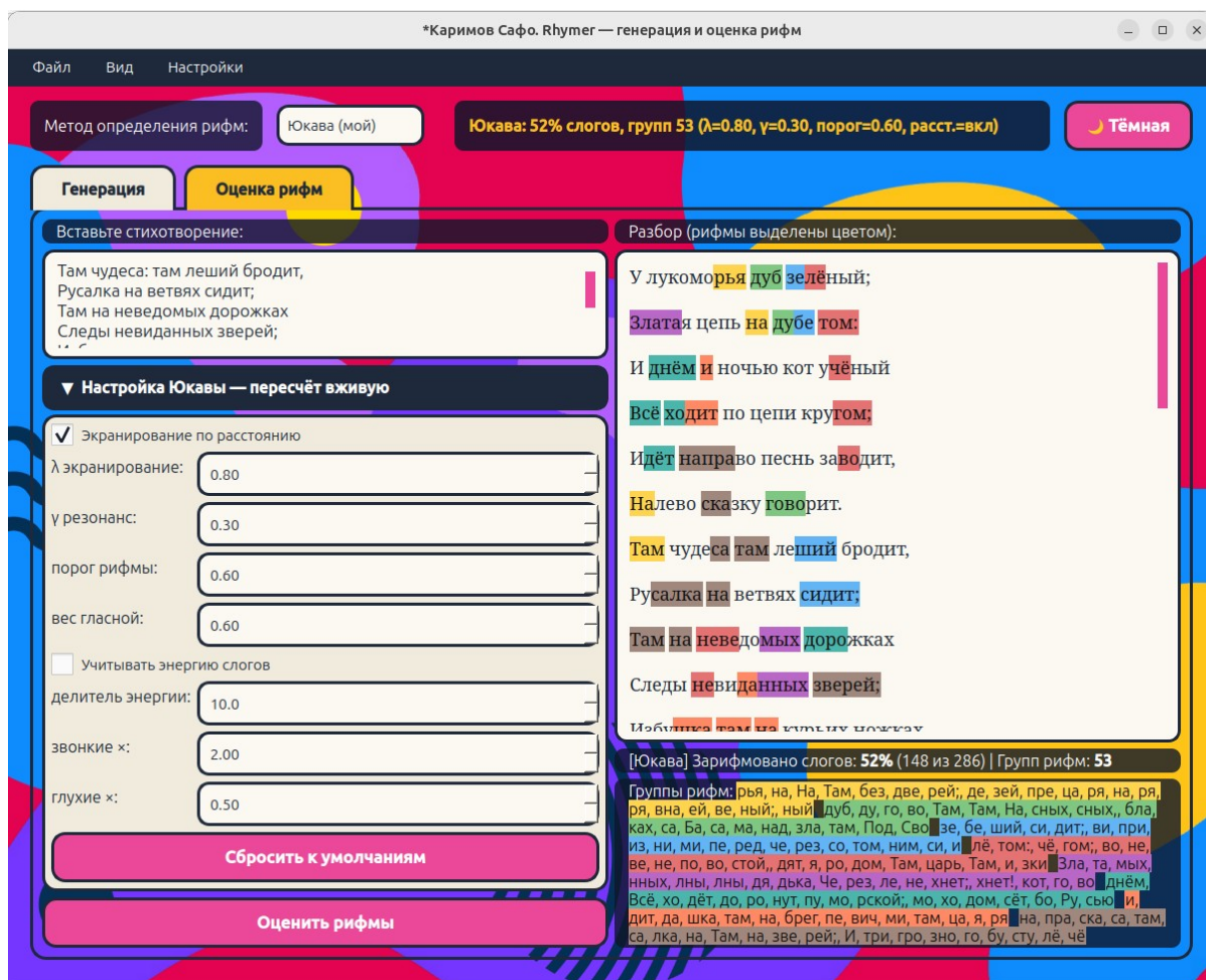


Рисунок А.5 — Оценка рифмы авторским детектором на потенциале Юкавы (группы рифм — компоненты связности графа)

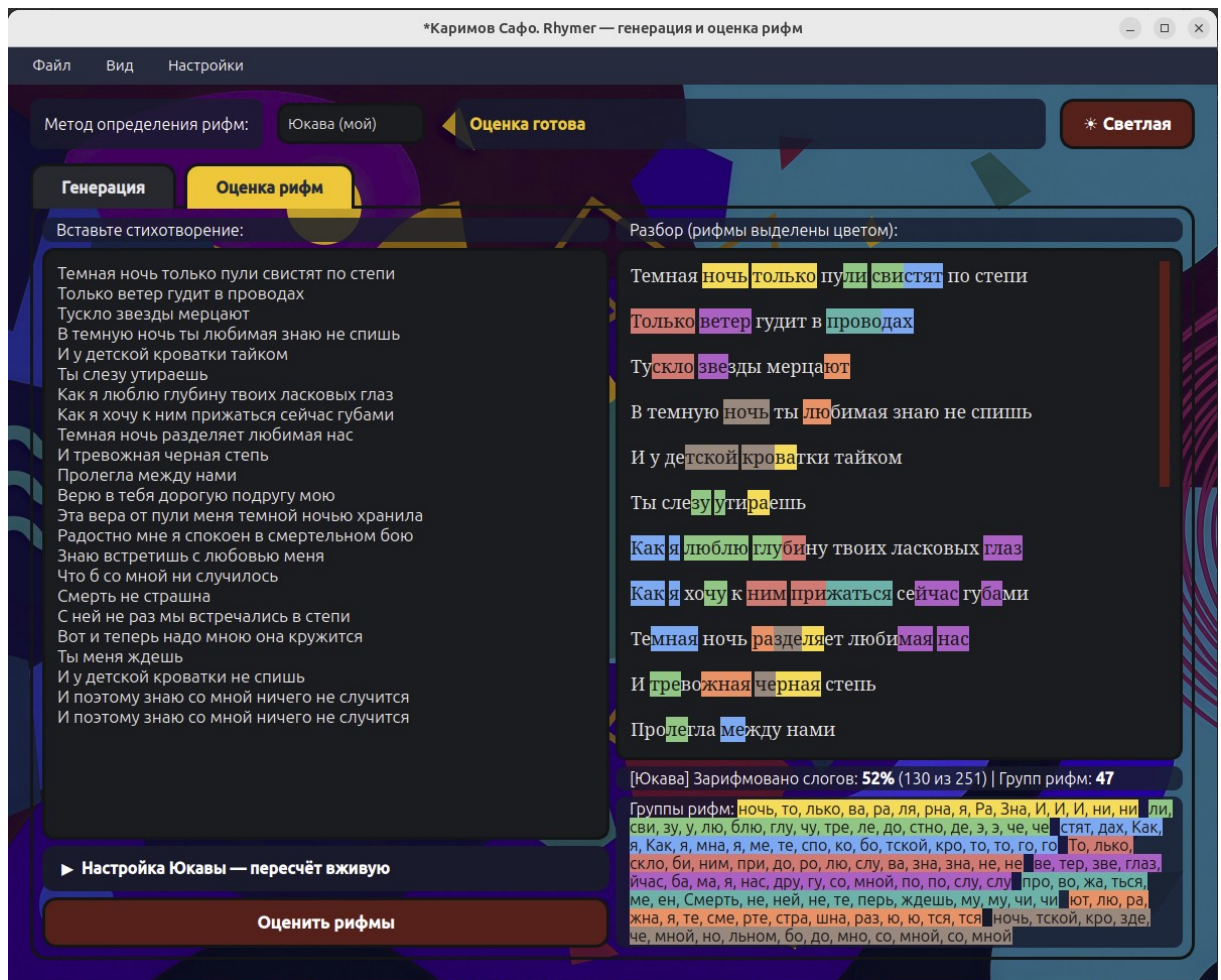


Рисунок А.6 — Тёмная тема интерфейса

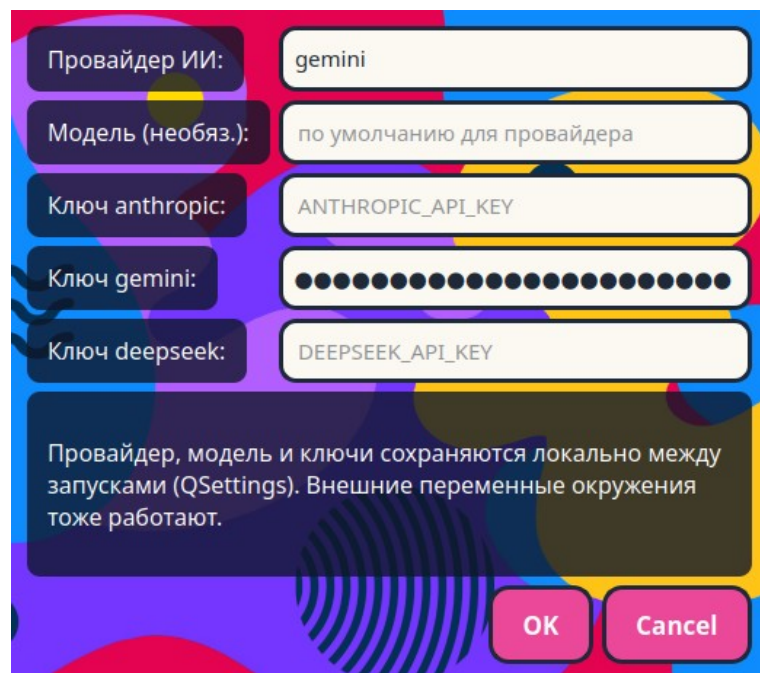


Рисунок А.7 — Настройки провайдера и API-ключей (Claude, Gemini, DeepSeek)

ПРИЛОЖЕНИЕ Б. Ключевые фрагменты исходного кода

(справочное)

Приведены ключевые фрагменты прототипа: ядро авторского детектора рифмы на потенциале Юкавы, определение энергии слога с фонетической векторизацией и непосредственные промпты к большой языковой модели. Настраиваемые константы (λ , γ , веса) вынесены в параметры и опущены для краткости.

Ядро детектора Юкавы: экранирующее ядро подавляет дальние связи слогов, а резонанс усиливает созвучные цепочки за счёт общих «друзей» (тройные корреляции).

Листинг Б.1 — Экранирующее ядро Юкавы и резонанс связей

```
def yukawa_kernel(d, lam=0.8):
    """Экранированное взаимодействие: подавляет дальние связи."""
    if d == 0:
        return 1.0
    return np.exp(-lam * d) / d

def apply_resonance(S, gamma=0.3):
    """Тройные корреляции (ритм): усиливаем связи с общими «друзьями». """
    S_new = S.copy()
    N = S.shape[0]
    for a in range(N):
        for b in range(a + 1, N):
            resonance = 0.0
            for c in range(N):
                if c != a and c != b:
                    resonance += S[a, c] * S[b, c]
            resonance /= N
            S_new[a, b] += gamma * resonance
            S_new[b, a] = S_new[a, b]
    return S_new
```

Из энергий слогов, их фонетической схожести и экранирования расстоянием собирается матрица связей S ; группы рифм выделяются как компоненты связности графа при пороге как доле от максимума связи.

Листинг Б.2 — Матрица связей слогов и выделение групп рифм

```
# узлы графа – слоги; nodes[k] = (строка_i, позиция_j, текст_слога)
energy = [syllable_energy(t, voiced_mult, voiceless_mult)
          for (_, _, t) in nodes]
S = np.zeros((N, N))
```

```

for a in range(N):
    for b in range(a + 1, N):
        d = manhattan_distance(nodes[a][:2], nodes[b][:2])
        sim = syllable_similarity(nodes[a][2], nodes[b][2], vowel_weight)
        k = yukawa_kernel(d, lam) # экранирование
расстоянием
        ef = abs(energy[a] * energy[b]) / energy_div # вклад «масс»
слогов
        S[a, b] = S[b, a] = sim * k * ef
S = apply_resonance(S, gamma=gamma)
S = S / S.max() # нормировка

# рифмы = компоненты связности графа при пороге link_ratio * max(S)
thr = link_ratio * off.max()
for a in range(N):
    for b in range(a + 1, N):
        if off[a, b] >= thr:
            parent[find(a)] = find(b) # union-find: объединяем созвучные
слоги

```

Энергия («масса») слога складывается из сонорности гласной и вклада согласных (звонкие тяжелее глухих).

Листинг Б.3 — Определение энергии слога (syllable energy)

```

def syllable_energy(syllable, voiced_mult=2.0, voiceless_mult=0.5):
    """«Масса» слога: основа – сонорность гласной, согласные добавляют
    массу (звонкие тяжелее глухих)."""
    syl = syllable.lower()
    base = 0.0 # сонорность гласной (основной тон)
    mass = 0.0 # вклад согласных
    for ch in syl:
        cls = VOWEL_SOUND.get(ch)
        if cls is not None:
            base = VOWEL_SONORITY[cls]
        else:
            f = CONS_FEATURES.get(ch)
            if f is not None:
                mass += voiced_mult if f[0] else voiceless_mult
    return base * (1.0 + mass)

```

Фонетическая векторизация: гласная задаёт основной тон (класс звука плюс артикуляция), согласные кодируются «мешком» признаков; схожесть слогов — взвешенный косинус блоков гласной и согласных.

Листинг Б.4 — Фонетическая векторизация и схожесть слогов

```

def syllable_vector(syllable):
    """Слог -> (вектор гласной, вектор согласных)."""
    syl = syllable.lower()
    vvec = np.zeros(len(VOWEL_CLASSES) + 3)
    cvec = np.zeros(CONS_DIM + 1)
    soft = 0.0
    for ch in syl:

```

```

        cls = VOWEL_SOUND.get(ch)
        if cls is not None:
            # гласная: класс +
            артикуляция
            vvec[VOWEL_CLASSES.index(cls)] = VOWEL_CLASS_WEIGHT
            vvec[len(VOWEL_CLASSES):] = np.multiply(VOWEL_ARTIC[cls],
                                                    VOWEL_ARTIC_WEIGHT)

            if ch in SOFTENING_VOWELS:
                soft = 1.0
            elif ch == 'ь':
                soft = 1.0
            else:
                f = CONS_FEATURES.get(ch)
                # согласные: «мешок»
                признаков
                if f is not None:
                    cvec[:CONS_DIM] += f
            cvec[CONS_DIM] = soft * SOFT_WEIGHT
        return vvec, cvec

def syllable_similarity(s1, s2, vowel_weight=0.6):
    """Схожесть = vowel_weight*cos(гласные) + (1-
    vowel_weight)*cos(согласные)."""
    v1, c1 = syllable_vector(s1)
    v2, c2 = syllable_vector(s2)
    return vowel_weight * _cosine(v1, v2) + (1.0 - vowel_weight) *
    _cosine(c1, c2)

```

Взаимодействие с ИИ построено как цикл «генерация — оценка — коррекция». Системный промпт и запрос best-of-N:

Листинг Б.5 — Системный промпт и промпт генерации (best-of-N)

```

SYSTEM = (
    "Ты — русский поэт. Сочиняешь силлабо-тонические рифмованные стихи. "
    "Строго соблюдаешь заданные тему, стихотворный размер, число строк и "
    "схему рифмовки. Отвечаешь только в запрошенном формате JSON, без
    пояснений."
)

# generate(): best-of-N — за один вызов модель выдаёт N разных вариантов
user = (
    f"Сочини {n} РАЗНЫХ вариантов стихотворения.\n"
    f"Тема: {theme}\n"
    f"Размер: {params.meter}\n"
    f"Число строк в каждом: {params.n_lines}\n"
    f"Схема рифмовки: {params.scheme}\n"
    f"Каждый вариант — ровно {params.n_lines} строк, строки разделяй '\n'.
    "
    f"Добивайся точной рифмы по схеме {params.scheme}."
)

```

Самокоррекция: модели передаётся разбор рифмы детектора с указанием, какие окончания сохранить, а какие переписать.

Листинг Б.6 — Промпт самокоррекции по обратной связи детектора

```
# improve(): переписать стих, улучшив рифму по разбору детектора
user = (
    "Вот стихотворение и разбор его рифмы:\n\n"
    f"СТИХ:\n{text}\n\n"
    f"РАЗБОР: {feedback}\n\n"
    "Перепиши стихотворение, улучшив рифму: СОХРАНИ хорошо рифмующиеся "
    "окончания, перепиши только плохие строки. Обязательно обеспечить: "
    "естественную читаемость (живой порядок слов, без неуклюжих инверсий "
    "ради рифмы), связный смысл, точную рифму по схеме "
    f"{params.scheme}, выдержанный размер {params.meter} и ровно "
    f"{params.n_lines} строк. Сохрани тему «{theme}»."
)
```